Contents

Multi-Phase, Multi-Wave Build Plan for Grok Build Tauri Control Panel

Aligned with Research Report Phases 0-4
Expanded with Full Multi-Agent Orchestration
Goal: Complete, error-free implementation via iterative agent loops.

Phase 0: Foundation & Discovery (Equivalent: 1-2 “days”)

Objective: Set up project skeleton, discover Grok Build binary/config, baseline state, initial crates.

Multi-Agent Waves:

Planning Wave (Parallel Planners): - Core Architect: Define Cargo workspace (grok_control_core, etc.), Tauri 2 setup (tauri.conf.json, src-tauri). - Tauri Integration Planner: Research latest Tauri 2 plugins (shell, fs, etc.), async runtime integration. - Security Planner: Define initial sandbox profiles, permission model from report. - Concurrency Planner: Design SessionRegistry with HashMap + tokio::Mutex or RwLock. - Output: Detailed specs, initial Cargo.toml, project dir structure, risk assessment (beta flux of Grok Build).

Implementation Wave (Parallel Implementers): - Implementer 1: Create workspace Cargo.toml with members. - Implementer 2: Basic Tauri app skeleton (main.rs with tauri::Builder, invoke handlers). - Implementer 3: grok_config crate: Serde structs for config.toml parsing. - Implementer 4: Simple CLI wrapper for `grok version` and `grok inspect --json`. - Deep Sketches: Full code in /code_sketches/phase0/

Audit Wave (Parallel Auditors): - All auditors review initial setup for completeness vs report, security in spawning, async readiness. - Reports: Any missing features? Crate version conflicts? Permission model gaps?

Revise Wave: - Fix any issues (e.g., add missing dependencies, refine structs). - Loop if needed (likely 1 loop).

Completion Criteria: Project compiles (cargo check), grok binary located, config parsed, baseline inspect output captured. All auditors PASS.

Advisory Documents: See /advisories/phase0_*.md

Phase 1: Core ACP & Single-Session Engine (1 week equivalent)

Objective: Implement ACP client, single long-lived session, basic tool event stream, plan mode.

Planning Wave: - ACP Specialist Planner: Full ACP flow spec (initialize, auth, session/new, prompt, stream updates). Map to Rust (tokio::process, serde_json for JSON-RPC). - Event Bus Planner: Design grok_events with broadcast channel for tool calls, plan updates. - Error Handling Planner: Graceful cancel, timeouts, error types (ACPError enum). - Outputs: ACP protocol diagram, data models (SessionId, ToolCallEvent), API for spawn_agent.

Implementation Wave (Parallel): - grok_acp Implementer: Full ACP client code (spawn, send initialize, handle responses, streaming decoder). - grok_control_core Implementer: SessionRegistry struct, AgentHandle, spawn_agent for ACP mode. - grok_events Implementer: Event bus, handlers for session/update. - Main

Tauri Implementer: Basic invoke commands for start_session, send_prompt, get_status. - Deep code: Full impls, examples of JSON-RPC over stdio.

Audit Wave: - Correctness Auditor: Verify ACP flow matches spec (from agentclientprotocol.com). - Concurrency Auditor: Check for race conditions in registry, proper Send/Sync. - Integration Auditor: Ensure headless fallback works. - Reports with line numbers.

Revise Wave: - Fix issues (e.g., add proper error propagation, fix streaming parser). - Re-audit until clean.

Completion: Single session works end-to-end in sketches (mock ACP responses), plan mode toggle, tool events streamed. Zero critical issues.

Phase 2: Multi-Session Orchestration & Worktrees

Objective: Multi-session concurrent, worktree support, permission presets, sessions list/export.

Planning Wave (Parallel): - Concurrency Planner: Multi HashMap<SessionId, AgentHandle>, worktree management (git commands via CLI wrapper). - Worktree Specialist: Lifecycle (create -w, list, rm, gc), landing changes (merge helpers). - Permission Planner: Global/per-session/per-tool rules, –always-approve toggle, sandbox profiles. - Outputs: Concurrency model, WorktreeManager trait/impl, PermissionController.

Implementation Wave: - Multiple Implementers for: Session Orchestrator (list/resume/fork/cancel), Worktree Manager, Permission & Sandbox Controller. - Update SessionRegistry for concurrent access (Arc<RwLock<... » or DashMap). - Deep sketches: Full code for worktree subcommands wrapper, git integration helpers.

Audit Wave: - Performance Auditor: Check for lock contention in multi-session. - Security Auditor: Validate sandbox + permission model (no escape). - Concurrency Auditor: Verify no deadlocks, proper isolation per worktree.

Revise Loop: Until all pass.

Completion: Multiple sessions simulatable, worktrees created/ managed, permissions enforced in code.

Phase 3: Extensions, MCP, Skills, Memory, Scheduler

Objective: Plugins/MCP/skills CRUD, dashboard queries, scheduler, memory, Imagine bridge, import/export.

Planning Wave: - Extensions Planner: CRUD for MCP (wrap grok mcp), plugins, skills. TOML rewrite + inspect. - Memory & Scheduler Planner: Memory service (flush/dream), scheduler (tokio::time or cron-like), /loop / /goal emulation. - Integrations Planner: Imagine proxy, Claude import, export Markdown, share URLs. - Outputs: API specs for Config & Extensions Browser, MemoryService, Scheduler.

Implementation Wave (Parallel Implementers): - Config & Extensions Implementer: Full CRUD logic, marketplace install (CLI wrap). - Memory Service Implementer. - Scheduler / Routines Implementer (tokio tasks). - Imagine Bridge & Import/Export Implementer. - Update event bus for new events.

Audit Wave (Specialized): - Maintainability Auditor: Code for extensions clean? - Integration Auditor: MCP toggle via /mcps equivalent. - Completeness: All report features covered?

Revise: Fix and loop.

Completion: All extensions functional in sketches, scheduler runs recurring jobs, memory persists.

Phase 4: Polish, Integrations, Testing, Deployment

Objective: Diff capture, advanced sandbox, import from Claude, polish, crash recovery, final docs.

Planning Wave: - Polish Planner: Diff engine (capture clean diffs post-plan), advanced sandbox (OS containers), crash recovery (persistence). - Testing Planner: Simulated tests, integration tests in sketches. - Deployment Planner: Bundle grok binary? Tauri build config, icons, etc. (backend focus).

Implementation Wave: - Polish Implementers: Diff capture logic, OS sandbox wrappers (conditional), recovery mechanisms. - Final glue code, comprehensive examples.

Audit Wave: - All auditors final comprehensive review. - Performance: Overall app responsiveness. - Security: Full model.

Revise Loop: Until zero issues.

Global Final Wave: Cross-phase audit, revise any lingering issues. Generate final docs.

Overall Completion: Full backend matches report 100%. All code sketches compile conceptually. Multi-agent loops exhausted errors. Ready for user to `cargo build` and extend with frontend.

Agent Workload Distribution (Maximizing Simultaneous Run)

- Peak simultaneous: 25 agents (e.g., 7 planners + 9 implementers + 7 auditors + 2 revisers in one wave).
- Orchestrator monitors via "progress files".
- In practice: Script to parallelize Grok calls or use local agent swarm.

This plan ensures **complete, robust build** leveraging Grok to its fullest.